https://github.com/mjvdheever/DSA2_Coursework

Marnitz van den Heever - DSA2 Final Assignment Submition

Import required libraries to execute program succesfully

In [1]:
```
#import required libraries
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from ipywidgets import interact
from matplotlib.animation import FuncAnimation, PillowWriter
from IPython.display import display
```

Task 1 Backtracking and Las Vegas Algorithms:

Coding Question1a

Load maze image and view image mode

Convert maze image into a numpy array and determine shape of array

In [2]:
```
#import maze(1).png
#matplotlib documentation recommends using PIL.image instead of matplotlib.image.imread
#Read an image from a file into a numpy array
img_path = "maze(1).png"

#Confirm format of image
print(f"Format of image loaded: {Image.open(img_path).mode}")

maze_1 = np.array(Image.open(img_path))

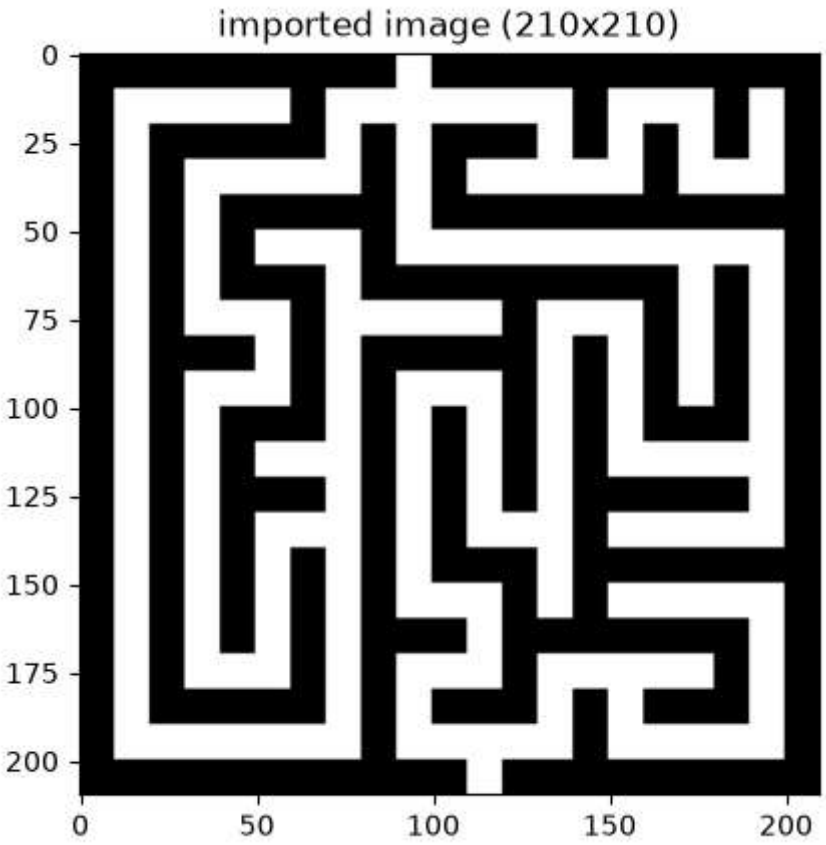
#imported image converted to numpy array - display the array
#print(maze_1)

#output number of rows, columns and colour channels
print(f"Shape of maze(1).png as numpy array: {maze_1.shape}")


#[255, 255, 255, 255] indicates a white pixel, i.e. pathway
#[0, 0, 0, 255] indicates a black pixel, i.e. wall
#Since image is black and white, we convert it to a binary matrix

#display the imported image
plt.imshow(maze_1)
plt.title(f"imported image ({maze_1.shape[1]}x{maze_1.shape[0]})")
plt.show()
```

```
Format of image loaded: RGBA
Shape of maze(1).png as numpy array: (210, 210, 4)
```



Initial loaded numpy array is size of 210 x 210
It's required to resize the array to 21 x 21

Determine step size divide factor to resize the array
Apply step size divide factor to resize array to array

In [3]:
```
#step size = ((210-1)-0)/(21-1) = 10.45

#determine the current height (rows) and width (columns) of the imported image
height = maze_1.shape[0]
width = maze_1.shape[1]

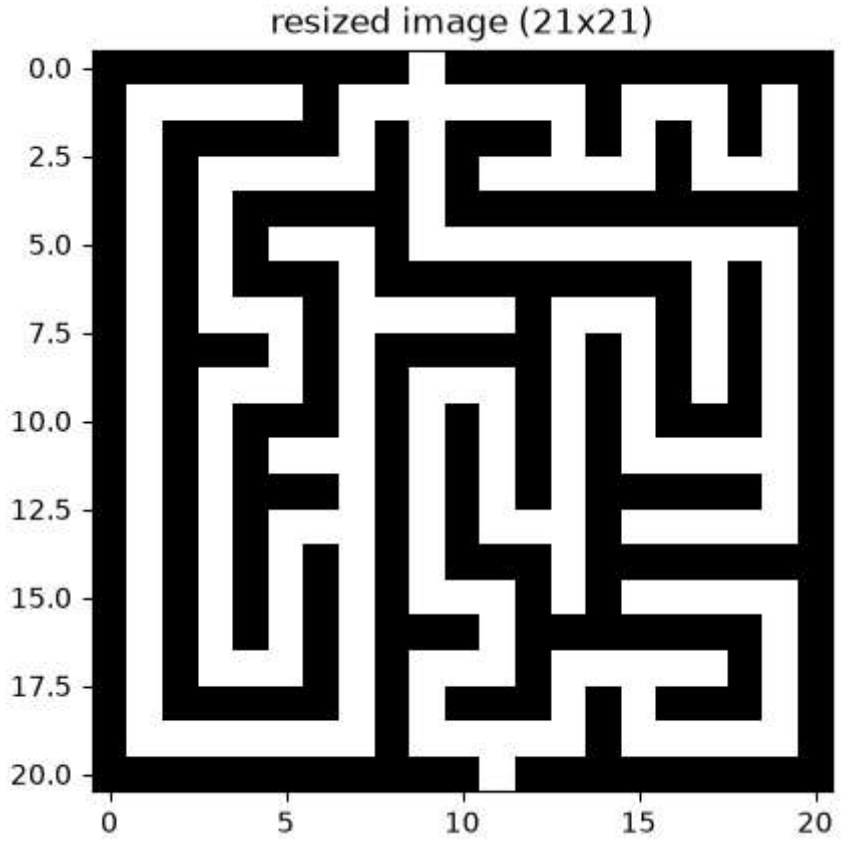
#array start at index 0 hence the -1 for the starting index
#force type as integer - Rounds down to the nearest integer
row_i = np.linspace(0, height - 1, 21).astype(int)
col_i = np.linspace(0, width - 1, 21).astype(int)

#resize the image to our 21x21 condition
maze_1_21_21 = maze_1[np.ix_(row_i, col_i)]

#Calculate height and width of resized matrix
height_21_21 = maze_1_21_21.shape[0]
width_21_21 = maze_1_21_21.shape[1]

#display resized image
plt.imshow(maze_1_21_21)
plt.title(f"resized image ({maze_1_21_21.shape[1]}x{maze_1_21_21.shape[0]})")
plt.show()
```

```
print(f"Height of resized image: {height_21_21}")
print(f"width of resized image: {width_21_21}")
```




```
Height of resized image: 21
width of resized image: 21
```

Convert resized image to binary matrix where:

1 = wall

0 = path

Given the RGBA PNG colour array, we can test if any Red/Blue/Green colour is omitted:

If any colour is present set to 1 (wall)

Iif no colour is omitted set 0 (pathway)

In [4]:
```
maze_grid = np.where(maze_1_21_21[:, :, 0] == 0, 1, 0)
print(maze_grid)
print(f"Shape of binary matrix: {maze_grid.shape}")
```

```
[[1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1]
 [1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 1]
 [1 0 1 1 1 1 1 0 1 0 1 1 1 0 1 0 1 0 1 0 1]
 [1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 1]
 [1 0 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1]
 [1 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1]
 [1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 1 0 1 0 1 1]
 [1 0 1 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0 1 0 1]
 [1 0 1 1 1 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 1]
 [1 0 1 0 0 0 1 0 1 0 0 0 1 0 1 0 1 0 1 0 1]
 [1 0 1 0 1 1 1 0 1 0 1 0 1 0 1 1 1 0 1 0 1]
 [1 0 1 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 1]
 [1 0 1 0 1 1 1 0 1 0 1 0 1 0 1 1 1 1 1 0 1]
 [1 0 1 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1]
 [1 0 1 0 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1]
 [1 0 1 0 1 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0 1]
 [1 0 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 0 1 1]
 [1 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0 1]
 [1 0 1 1 1 1 1 0 1 0 1 1 1 0 1 0 1 1 1 0 1]
 [1 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1]
 [1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1]]
Shape of binary matrix: (21, 21)
```

Dynamically find the entrance/exit of the maze - return the element(s) position of entrance/exit

Assignment specifies that the starting point of the maze is at the top row:

Assign top row as entrance

Assign bottom row as exit

In [5]:
```python
#Find the entrance/starting point of the binary maze, return the element(s)
#Ring fence only the first/last row and column
entrances = []

#Find the elements in the binary maze with value of 0, if 0 then append to entrances array
for row, col in np.argwhere(maze_grid == 0):
    if row in (0, maze_grid.shape[0] - 1) or col in (0, maze_grid.shape[1] - 1):
        entrances.append((int(row), int(col)))

print(f"Binary maze entrance points: {entrances}")

#Assignment allocates the entrance to the top row, so exist subsequently must be last row

starting_point=entrances[0]
print(f"Binary maze starting point: {starting_point}")
exit_point=entrances[1]
print(f"Binary maze exit point: {exit_point}")
```

```
Binary maze entrance points: [(0, 9), (20, 11)]
Binary maze starting point: (0, 9)
Binary maze exit point: (20, 11)
```

Create modular functions that will be used in backtracking and Las Vegas approach

Assignment specificies allowed movements (no diagonal movements allowed) - include these conditions as part of initial setup

In [6]:

```python
#Determine movements at a point provided
def check_movements(matrix,point):
    n_values =[]
    row,col=point

    for r_move, c_move in movements:
        row_get = row + r_move
        col_get = col + c_move

        if 0<=row_get<=(height_21_21-1) and 0<=col_get<=width_21_21-1:
            value = int(matrix[row_get, col_get])
            n_values.append(((row_get,col_get),value))
    return n_values

#Determine if any adjcent elements contain a wall and what moves are valid
def valid_movements_check(movements):
    valid_movements = [points[0] for points in movements if points[1] == 0]
    return valid_movements

#Generate a function to keep track of unique movements to enable backtracking conditions
#Set starting point as the first movement in movement_log
movement_log = [starting_point]
def movement_tracker(movement_log, point):
    if point not in movement_log:
        movement_log.append(point)
    return movement_log

#Identify what unexplored moves are available for backtracking conditions
def unexplored_movements(valid_moves, movement_log):
    unexplored = [point for point in valid_moves if point not in movement_log]
    return unexplored

#Choose first available movement if possible and not explored
#Set start point as the first point in the path explored
current_path = [starting_point]
def next_movement(unexplored, current_path):
    if unexplored:
        next_point = unexplored[0]
        current_path.append(next_point)
        return next_point

    #If no unexplored movements exist, backtrack
    else:
        print("No next movements available - backtrack one step")

        #Go one step back and remove the current position from the path
        current_path.pop()

        #Return to the previous position
        if current_path:
            return current_path[-1]

        #No movements remaining to explore
        return None

#For Las Vegas approach - generate a random next movement with wall constraints
def random_movement(valid_moves):
```

```
    #Check if any movements are available
    if not valid_moves:
        return None

    #Randomly select a valid movement
    random_index = np.random.randint(0, len(valid_moves))
    next_move = valid_moves[random_index]

    return next_move
```

Backtracking function

Coding question 1.b.ii

We predfine the movements based on the assignment criteria, where the only allowed movements are up/down/left/right as stored in the **movements** *list*

Before taking a next step, the backtracking function will first determine what movements are avaible by calling the **check_movements** *and* **valid_movements_check** *functions. These function ensure the next possible move is either up/down/left/right and not moving outside of the matrix, i.e. staying within the borders of the maze. Hence only keeping positions where movement is possible*

Since we have defined **movements** *as* **[(-1, 0), (1, 0), (0, -1), (0, 1)]**, *the function will first check if going up is possible, if not it will check if down movement is possible,followed by left and lastly right.*

The **movement_tracker** *function stores and keeps track of the unique locations visited as we move through the matrix, by using the* **movement_log**.

The **unexplored_movements** *function will check if any of the results from* **valid_movements_check** *has been visited yet, if it has not been visisted yet, assign it to the* **unexplored** *list, this ensures that previously visisted locations are not selected again when moving forward with a next movement.*

The **next_movement** *function will take the element from* **unexplored** *array and assign it as the next location to move to. The selected location is also added to* **current_path** *which keeps track of the current route taken so far .*

If no movements are available, i.e. **unexplored** *is any empty list, then pop the last element from* **current_path**, *essentially removing the last position from the list and moving one step back, which follows a Last-In, First-Out approach. In the latter case, we have added the location to the* **movement_log** *so this location can not be visited again when calling the* **unexplored_movements** *function*

This process will repeat with the loop in place until either the exit point is reached or if we no more unexplored options. Having an empty **unexplored** will stop the algorithm since we will first attempt to move one step back to previous location, only when* **current_path** becomes an empty list then* **next_movement** *will return a null/none value, in which case the loop will break.*

This approach follows a depth-first search as observed in DSA-1, where it explores a branch as far as possible before moving back to explore other branches

Use modular functions as defined above

In [7]:
```
#Possible movement (up/down/left/right) at any given point
movements = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def backtracking(matrix,starting_point,exit_point):
```

```
    #Set initial conditions
    movement_log = [starting_point]
    movement_all_log = [starting_point]
    current_path = [starting_point]
    current_point = starting_point
    moves=0
    success_count = 0

    while current_point != exit_point:

        #Step1 Find possible movements
        possible_moves = check_movements(matrix, current_point)

        #Step2 Find valid movements
        valid_moves = valid_movements_check(possible_moves)

        #Step3 Remove explored movements
        unexplored = unexplored_movements(valid_moves, movement_log)

        #Step4 Select next movement
        previous_path_length = len(current_path)
        next_point = next_movement(unexplored, current_path)
        if len(current_path) < previous_path_length:
            print(f"Backtracking to {next_point}")
            current_point = next_point

        #Stop if no movements remain
        if next_point is None:
            break

        #Step5 Log movement
        movement_all_log.append(next_point)
        movement_tracker(movement_log, next_point)

        if current_point != next_point:
            print(f"current point: {current_point} and move to next point {next_point}")
        #Update current point
        current_point = next_point
        moves=moves+1

        if current_point == exit_point:
            success_count += 1

    return {
    "moves": moves,
    "movement_log": movement_log,
    "movement_all_log": movement_all_log,
    "current_path": current_path,
    "path_length": len(current_path),
    "success_count": success_count,
    }
```

Las Vegas function

Coding question 1.b.i

The las_vegas function will attempt 200 movements or stop if the exit point is reached before 200 moves - as seen in the while loop process in the function below

Use modular functions as defined above

In [8]:
```
def las_vegas(matrix, starting_point, exit_point):

    #Set initial conditions
    movement_log = [starting_point]
    movement_all_log = [starting_point]
    current_point = starting_point
    moves = 0
    success_count = 0

    #Max 200 moves allowed or until exist is found
    while current_point != exit_point and moves < 200:

        #Step1 Find possible movements
        possible_moves = check_movements(matrix, current_point)

        #Step2 Find valid movements
        valid_moves = valid_movements_check(possible_moves)

        #Step3 Randomly select next movement with wall conditions applied
        next_point = random_movement(valid_moves)

        #Stop if no valid movements exist
        if next_point is None:
            break

        #Step4 Log movement
        movement_all_log.append(next_point)
        movement_tracker(movement_log, next_point)

        #Step5 Update current position
        current_point = next_point
        moves += 1

    #Keep track successful exists
    if current_point == exit_point:
        success_count += 1

    return {
        "success_count": success_count,
        "moves": moves,
        "movement_log": movement_log,
        "movement_all_log": movement_all_log
    }
```

Generate movement GIF function

In [9]:
```
#Function to generate an animation to show the movements of the algorithm
def generate_maze_gif(matrix, movement_all_log, algorithm, starting_point, exit_point, fps=4):

    history = np.array(movement_all_log)
```

```
    #create/Overwrite file
    gif_filename = f"maze_{algorithm.lower().replace(' ', '_')}.gif"

    #Generate maze
    fig, ax = plt.subplots(figsize=(8, 8))
    ax.imshow(matrix, cmap="gray_r")

    #Create movement points
    trail, = ax.plot([], [], "b.-", linewidth=2,markersize=4, label="Movement history")

    current_dot, = ax.plot([], [], "ro", markersize=8,label="Current position")

    #Indicate entrance and exit
    ax.plot(starting_point[1], starting_point[0],"go", markersize=9, label="Entrance")

    ax.plot(exit_point[1], exit_point[0],"mo", markersize=9, label="Exit")

    ax.set_xticks([])
    ax.set_yticks([])
    ax.legend()

    #Plot movement point for each frame
    def update(frame):
        path_so_far = history[:frame + 1]

        #Columns
        x = path_so_far[:, 1]
        #Rows
        y = path_so_far[:, 0]

        trail.set_data(x, y)
        current_dot.set_data([x[-1]], [y[-1]])

        ax.set_title(f"{algorithm} - Movement {frame}")

        return trail, current_dot

    #Generate gif for each frame
    gif = FuncAnimation(
        fig,
        update,
        frames=len(history),
        interval=1000 / fps,
        blit=False,
        repeat=False
    )


    gif.save(gif_filename, writer=PillowWriter(fps=fps))
    plt.close(fig)

    return gif_filename
```

User input

Coding question1b

Please select which approach to apply in an attempt to exit the maze:

If you want to change the approach, please select a different option from the drop down list and run the processing step

In [10]:
```
#ask user what approach they want to take
#Use drop down list to avoid the user selecting an option that is not in scope - we will ipywidge
def method(user_method=["Backtracking", "Las Vegas"]):
    global method
    method = user_method
    print(f"Active Method: {method}")

interact(method);
```

```
interactive(children=(Dropdown(description='user_method', options=('Backtracking', 'Las Vegas'),
value='Backtr…
```

Algorithm processing step

Please run this step after selecting the method

In [95]:
```
#confirm that the method/approach is captured correctly when a new selection is made
total_runs = 1

if method == "Backtracking":
    result = backtracking(maze_grid, starting_point, exit_point)

elif method == "Las Vegas":
    result = las_vegas(maze_grid, starting_point, exit_point)

print("Successful attempts:", result["success_count"])
print("Total movements:", result["moves"])
print("Unique positions:", len(result["movement_log"]))
print("Total runs:", total_runs)

from IPython.display import Image as NotebookImage, display
gif_file = generate_maze_gif(
    maze_grid,
    result["movement_all_log"],
    method,
    starting_point,
    exit_point
)

display(NotebookImage(filename=gif_file,embed=True))
```

```
Successful attempts: 0
Total movements: 200
Unique positions: 17
Total runs: 1
<IPython.core.display.Image object>
```

To view the visited square for each algorithm please open gif links below:

Coding question 1.c

Backtracking: https://github.com/mjvdheever/DSA2_Coursework/blob/main/maze_backtracking.gif

Las Vegas: https://github.com/mjvdheever/DSA2_Coursework/blob/main/maze_las_vegas.gif

Testing

Errors encountered

Error1

Error type: Syntax error - missing quotation mark when calling print function

Error impact: caused program to fail - unable to execute

Fix applied: Added missing quotation mark at the end on the print statement

In [14]:
```
import matplotlib.image as mpimg
error1 = mpimg.imread('Error1.jpg')
plt.axis('off')
plt.imshow(error1)
```

Out[14]:
```
<matplotlib.image.AxesImage at 0x261624eca50>
```



```
In [34]:
#import maze(1).png
#matplotlib documentation recommends using PIL.Image instead of matplotlib.image.imread
#Read an image from a file into a numpy array
img_path = "maze(1).png"
maze_1 = np.array(Image.open(img_path))

#imported image converted to numpy array - display the array
print(maze_1)

#output number of rows, columns and colour channels
print(f"Shape of maze(1).png as numpy array: {maze_1.shape}")

#Confirm format of image
print(f"Format of image loaded: {Image.open(img_path).mode}")

#[255, 255, 255, 255] indicates a white pixel, i.e. pathway
#[0, 0, 0, 255] indicates a black pixel, i.e. wall
#Since image is black and white, we convert it to a binary matrix

#display the imported image
plt.imshow(maze_1)
plt.title(f"Imported image ({maze_1.shape[1]}x{maze_1.shape[0]})")
plt.show()

  Cell In[34], line 14
    print(f"Format of image loaded: {Image.open(img_path).mode})
                                                               ^
SyntaxError: unterminated f-string literal (detected at line 14)
```

Error2

Error type: Logic error - Attempting to dynamically find the entrances of the matrix. Initial attempt returned locations of all elements with value of 0

Error impact: Caused program to select incorrect starting point - backtracking and Las Vegas algorithms would not execute correctly

Fix applied: Applied logic to only look for 0 values in the first row, last row, first column and last column with code below:

In [15]:
```
#for row, col in np.argwhere(maze_grid == 0):
#    if row in (0, maze_grid.shape[0] - 1) or col in (0, maze_grid.shape[1] - 1):
#        entrances.append((int(row), int(col)))'
```

In [16]:
```
import matplotlib.image as mpimg
error2 = mpimg.imread('Error2.jpg')
plt.axis('off')
plt.imshow(error2)
```

Out[16]:
```
<matplotlib.image.AxesImage at 0x261624eefd0>
```



Error3

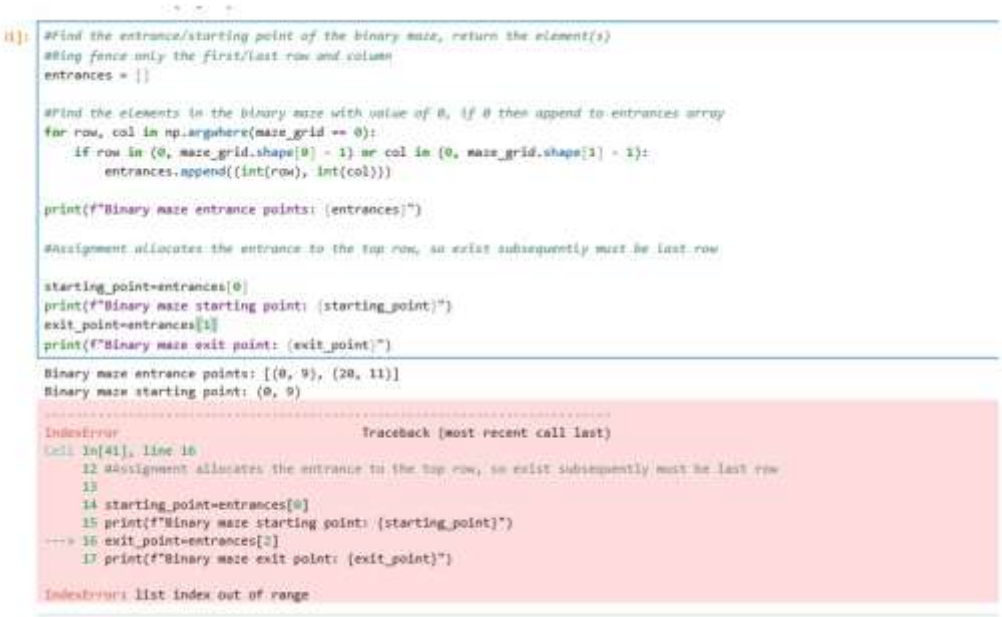
Error type: index error - Attempting to dynamically find the exit of the matrix. Initial attempt referenced items that are outside of the range of the list since index starts at [0]

Error impact: Caused program to fail - unable to execute

Fix applied: Identified the list where the error originated from, adjusted the indexes of the list to reference the correct item: exit_point = entrances[1]

In [17]:
```
import matplotlib.image as mpimg
error3 = mpimg.imread('Error3.jpg')
plt.axis('off')
plt.imshow(error3)
```

Out[17]:
```
<matplotlib.image.AxesImage at 0x261624b1f90>
```



Error3

Error type: Logic error - Attempting to show the movements through the maze, including backtracking steps. Initial attempt did not show when the algorithm was backtracking

Error impact: No critical failure, however was unable to verify if backtracking executed correctly and subsequently backtracking was not correctly applied

Fix applied: Added a print statement to show when backtracking occured and where it moved to with the code below:

In [19]:
```
#if len(current_path) < previous_path_length:
    #print(f"Backtracking to {next_point}")
    #current_point = next_point
```

In [20]:
```
import matplotlib.image as mpimg
error5 = mpimg.imread('error5_backtracking not displayed.jpg')
plt.axis('off')
plt.imshow(error5)
```

Out[20]: <matplotlib.image.AxesImage at 0x2616240f890>



Error4

Error type: Logic error - Attempting to track all the movements through the maze, including backtracking steps. Initial attempt double counted back track movements

Error impact: No critical failure, however inaccurately counted the total number of movements to until the exit point was found

Fix applied: Removed incrementenal movement counter when back tracking and applied only at the last step of the loop:

In [21]:
```
#if len(current_path) < previous_path_length:
    #print(f"Backtracking to {next_point}")
    #current_point = next_point
    #moves += 1 #Removed this statement and moved it to the end of the loop
```

In [22]:
```
import matplotlib.image as mpimg
error6 = mpimg.imread('error6_movement_counter.jpg')
plt.axis('off')
plt.imshow(error6)
```

Out[22]: <matplotlib.image.AxesImage at 0x2616240e350>



Testing

Backtracking vs Las Vegas Comparison

What is the difference between Backtracking and Las Vegas
The main difference between the Backtracking and Las Vegas alogrithm is how the next movement is selected and how previously visited locations are handled. Both alogrithms have the same movement restrictions, being only able to move one step up/down/left/right and are not allowed to move outside of the maze border

Backtracking: The Backtracking approach follows a predefined order of movements, where the first available unexplored movement is selected based on the movements list, prioritising up, down, left and right (these are the defined movements in this specfic order).
This approach ensures that the algorithm explores the maze without repeatedly exploring the same locations.
The algorithm will continue until the exit is found or all possible routes have been explored.

Las Vegas: The Las Vegas approach does not follow a predefined movement with priority when selecting the next position. Instead it randomly selects one of the available movements (given the movement criteria of up/down/left/right and must stay within the border).
This randomized movement selection process can result in a location being explored multiple times and also allows the algorithm to explore the same movement patterns that was already explored. So this can result in the algorithm spending a large number of movements exploring the same area of the maze, rather than progressing towards the exit.
The Las Vegas approach also has a maximum limit of 200 movements (as specified in the assignment). The algorithm will be successful if the exit is reached within the 200 movements, otherwise it will be unsuccessful once the movement limit has been reached.

Which data structure/s would you use to implement this problem?

NumPy Array:
Converting an image to array - The maze is initially imported from the PNG image and then converted into a binary NumPy array.
The array stores the maze structure where 0 represents a walkable path and 1 represents a wall.
This also allows the functions to access specific locations within the maze using row and column as "coordinates".
NumPy array also enables to interpret the maze image as a two-dimensional grid/matrix which allows to identify explored-, next-, valid- and invalid movements

Python Lists/Tuples:
Python lists are used throughout both algorithms to store available movements, explored locations, unexplored locations and the sequence of steps as you move through the maze.
The movements list stores the predefined movement directions as tuples (in a specific order of (-1, 0), (1, 0), (0, -1), (0, 1)).
Lists were chosen since they are easy to update using functions such as append() and allows iterations and compare the available movements against previously visited positions.

Last-in, First-out Stacks:
For the Backtracking approach, the current_path list is used as a stack, following LIFO principle.
When moving to a next location, the next position is added to current_path using append(). If no unexplored movements are available, the function uses pop() to remove the most recently added position and move one step backwards.

Dictionary:
Both algorithms return the results using a Python dictionary, containing values such as succesful exits, all movements log and unique movements. This also makes it easier to compare results of both the backtracking and Las Vegas algorithm

Backtracking vs Las Vegas Comparison success rate after 10 000 runs

In [27]:
```python
#Generate function to compare succesful run comparison

import contextlib
import io

def test_success_rate(matrix, starting_point, exit_point, total_runs=10000):

    backtracking_successes = 0
    las_vegas_successes = 0

    for run in range(total_runs):

        #Avoid printing each movement
        with contextlib.redirect_stdout(io.StringIO()):
            #Backtracking
            backtracking_result = backtracking(matrix, starting_point, exit_point)

            #Las Vegas
            las_vegas_result = las_vegas(matrix, starting_point, exit_point)

        #Track succesful exists
        backtracking_successes += backtracking_result["success_count"]
        las_vegas_successes += las_vegas_result["success_count"]

    #success rates
    backtracking_rate = (backtracking_successes / total_runs) * 100
    las_vegas_rate = (las_vegas_successes / total_runs) * 100
```

```
    return {
        "total_runs": total_runs,
        "backtracking_successes": backtracking_successes,
        "las_vegas_successes": las_vegas_successes,
        "backtracking_rate": backtracking_rate,
        "las_vegas_rate": las_vegas_rate
    }

#Execute function
results = test_success_rate(maze_grid,starting_point,exit_point,total_runs=10000)



print("Backtracking performance:")
print("Total Backtracking runs:", results["total_runs"])
print("Successful Backtracking runs:", results["backtracking_successes"])
print(f"Backtracking success rate: {results['backtracking_rate']:.2f}%")
print("\n")
print("Las Vegas performance (max200 movements):")
print("Total Las Vegas runs:", results["total_runs"])
print("Successful Las Vegas runs:", results["las_vegas_successes"])
print(f"Las Vegas success rate: {results['las_vegas_rate']:.2f}%")
```

```
Backtracking performance:
Total Backtracking runs: 10000
Successful Backtracking runs: 10000
Backtracking success rate: 100.00%


Las Vegas performance (max200 movements):
Total Las Vegas runs: 10000
Successful Las Vegas runs: 0
Las Vegas success rate: 0.00%
```

Key observations:

Backtracking:

Since backtracking follows a predefined movement priority, the success rate for each run will be exactly same.

If the first run will be successfull then all consecutive runs will be succesfull as well, the opposite is also true, if the first run is unsucessfull then all consecutive runs will be unsucessfull.

In the this case, the first run was successfull so we have a 100% success rate for all 10 000 runs.

Las Vegas:

Las Vegas algorithm randomly selects the next available movement using the np.random.randint() function. This means the algorithm can repeatedly apply the exact same movement pattern without effectively "move forward".

Additional the run will be defined as unsuccesfull if it reaches the 200 movement limit

In this case of 10 000 runs, 0 runs executed successfully, resulting in a 0% success rate.

Based on this outcome, we can conclude that **backtracking** is the more effective algorithm to successfully exit the maze since it avoids repeating movement sequences.

Task 2 Minimum Spanning Tree (MST) algorithms:

Import required libraries to execute program succesfully

In [28]:
```
import networkx as nx
import matplotlib.pyplot as plt
import random
```

Coding question 2.a

In [32]:
```
#Set seed value to ensure rerunning is consistent
random.seed(1337)

#Create node connections
edges = [
    ("A", "B"),
    ("A", "C"),
    ("A", "D"),
    ("A", "E"),
    ("B", "D"),
    ("B", "E"),
    ("C", "D"),
    ("C", "E"),
    ("D", "E"),
    ("D", "F"),
    ("E", "F")
]

graph = nx.Graph()

#Generate random edge weights
for start, end in edges:
    weight = random.randint(1, 20)
    graph.add_edge(start, end, weight=weight)

#Check assignment node and edge requirements
print("Total nodes:", graph.number_of_nodes())
print("Total edges:", graph.number_of_edges())
print("Is the graph connected?:", nx.is_connected(graph))

#Plot graph
pos = nx.spring_layout(graph, seed=1337)

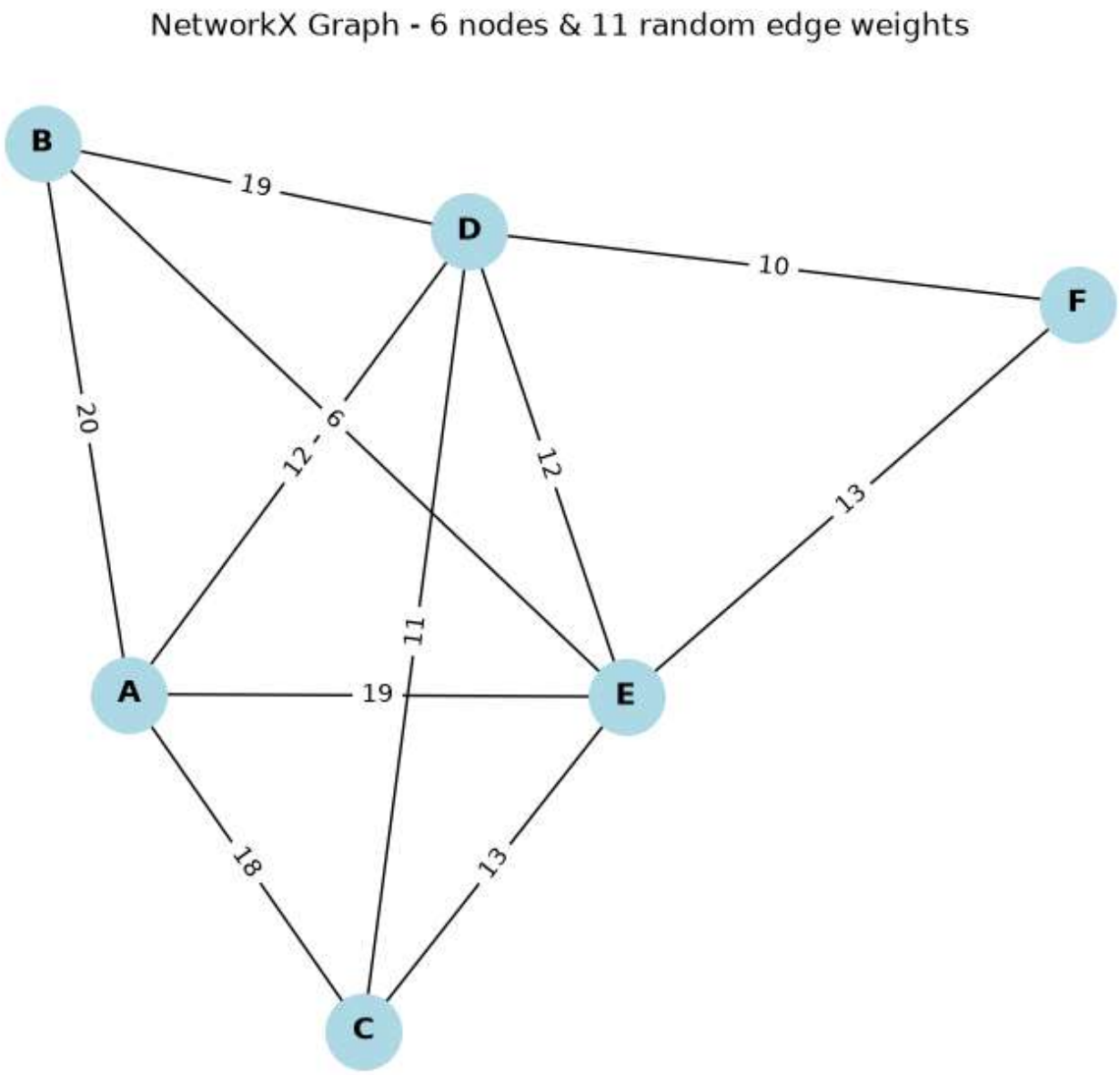
plt.figure(figsize=(7, 6))

nx.draw(graph,pos,with_labels=True,node_color="lightblue",node_size=900,font_weight="bold")

edge_weights = nx.get_edge_attributes(graph, "weight")
nx.draw_networkx_edge_labels(graph,pos,edge_labels=edge_weights)

plt.title("NetworkX Graph - 6 nodes & 11 random edge weights")
plt.show()
```

```
Total nodes: 6
Total edges: 11
Graph is connected: True
```



Coding 2.b.a: Kruskal's MST

Create Kruskal's MST algorithm (greedy algorithm)

Greedy algorithms works in a way by selecting the optimal choice at each immediate step, i.e. thinking "locally", considering only the next best step at the current point and not the best step for the bigger picture (not "globally")

Coding 2.b.b: Create function to execute Kruskal's MST algorithm

In [42]:
```python
def MyMinimumSpanningTree(Graph) -> nx.Graph:
    #Set an empty graph with our predefined nodes
    kruskal = nx.Graph()
    kruskal.add_nodes_from(Graph.nodes())
```

```
    #Sort edges from smallest to largest weights
    #NetworkX output structure (node_from,node_to,{weight: val}
    #NetworkX output structure - dictionary: first element[from_node], second element [to_node],
    sorted_edges = sorted(Graph.edges(data=True), key=lambda edge: edge[2]["weight"])

    #Create a loop to sort the edges from smallest to largest weights
    for step, (start, end, data) in enumerate(sorted_edges, start=1):
        weight = data["weight"]

        #Check if nodes are connected
        if nx.has_path(kruskal,start,end):
            #If already exists remove to avoid loop between same nodes
            print(f"Step {step}: {start} -> {end} with weight: {weight}  (remove)")
        else:
            kruskal.add_edge(start,end,weight=weight)
            print(f"Step {step}: {start} -> {end} with weight: {weight} (add)")

    return kruskal

kruskal_result = MyMinimumSpanningTree(graph)
```

```
Step 1: B -> E with weight: 6 (add)
Step 2: D -> F with weight: 10 (add)
Step 3: C -> D with weight: 11 (add)
Step 4: A -> D with weight: 12 (add)
Step 5: D -> E with weight: 12 (add)
Step 6: C -> E with weight: 13  (remove)
Step 7: E -> F with weight: 13  (remove)
Step 8: A -> C with weight: 18  (remove)
Step 9: A -> E with weight: 19  (remove)
Step 10: B -> D with weight: 19  (remove)
Step 11: A -> B with weight: 20  (remove)
```

Coding 2.c: Visualize Kruskal's MST

In [65]:

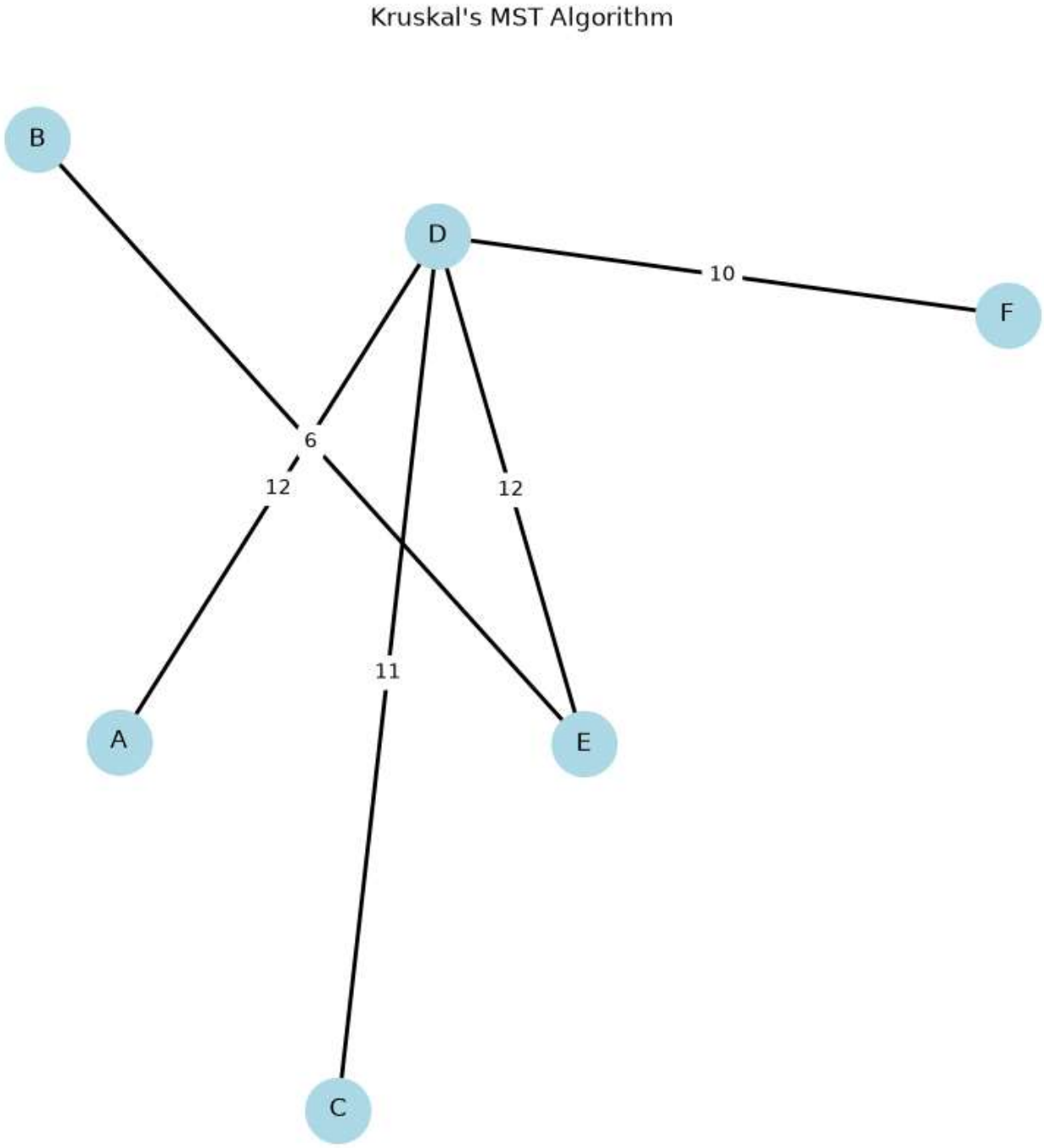
```
#Generate plot figure
plt.figure(figsize=(8, 8))

nx.draw(kruskal_result,pos,with_labels=True,node_color="lightblue",node_size=1000,width=2)

kruskal_weights = nx.get_edge_attributes(kruskal_result, "weight")

nx.draw_networkx_edge_labels(kruskal_result,pos,edge_labels=kruskal_weights,rotate=False)

plt.title("Kruskal's MST Algorithm")
plt.show()
```



Testing a: Validate Kruskal's MST with two graphs

In [74]:
```
#Generate Graph2 with 5 nodes and 8 edges
edges_graph2 = [
    ("A", "B"),
    ("A", "C"),
    ("A", "E"),
    ("B", "C"),
    ("B", "D"),
    ("B", "E"),
    ("C", "D"),
    ("D", "E")
]
```

```
#Generate Graph2 weights
graph2 = nx.Graph()
random.seed(2)
#Generate random edge weights
for start, end in edges_graph2:
    weight2 = random.randint(1, 23)
    graph2.add_edge(start, end, weight=weight2)

#Check assignment node and edge requirements
print("Total nodes:", graph2.number_of_nodes())
print("Total edges:", graph2.number_of_edges())
print("Is the graph connected?:", nx.is_connected(graph2))

#Plot graph
pos2 = nx.spring_layout(graph2, seed=2)

plt.figure(figsize=(7, 6))

nx.draw(graph2,pos2,with_labels=True,node_color="lightblue",node_size=900)

weight2 = nx.get_edge_attributes(graph2, "weight")
nx.draw_networkx_edge_labels(graph2,pos2,edge_labels=weight2,rotate=False)

plt.title("NetworkX Graph2 - 5 nodes with 8 random edge weights")
plt.show()

#Generate Graph 3 with 7 nodes and 10 edges
edges_graph3 = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "E"),
    ("C", "F"),
    ("D", "E"),
    ("D", "G"),
    ("E", "G"),
    ("G", "F")
]

#Generate Graph3 weights

graph3 = nx.Graph()
random.seed(3)
#Generate random edge weights
for start, end in edges_graph3:
    weight3 = random.randint(10, 24)
    graph3.add_edge(start, end, weight=weight3)

#Check assignment node and edge requirements
print("Total nodes:", graph3.number_of_nodes())
print("Total edges:", graph3.number_of_edges())
print("Is the graph connected?:", nx.is_connected(graph3))

#Plot graph
pos3 = nx.spring_layout(graph3, seed=3)

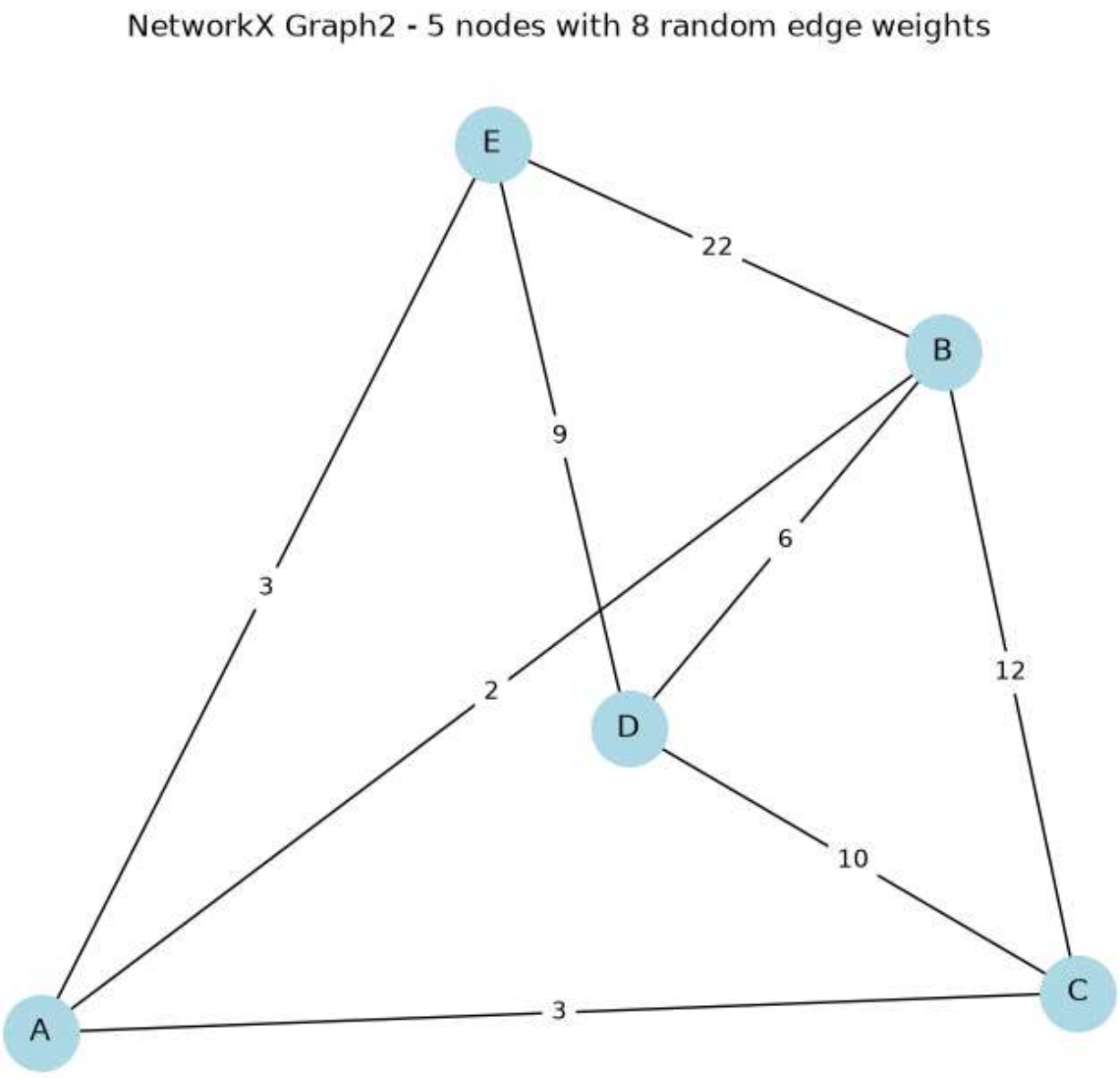
plt.figure(figsize=(7, 6))
```

```
nx.draw(graph3,pos3,with_labels=True,node_color="lightblue",node_size=900)

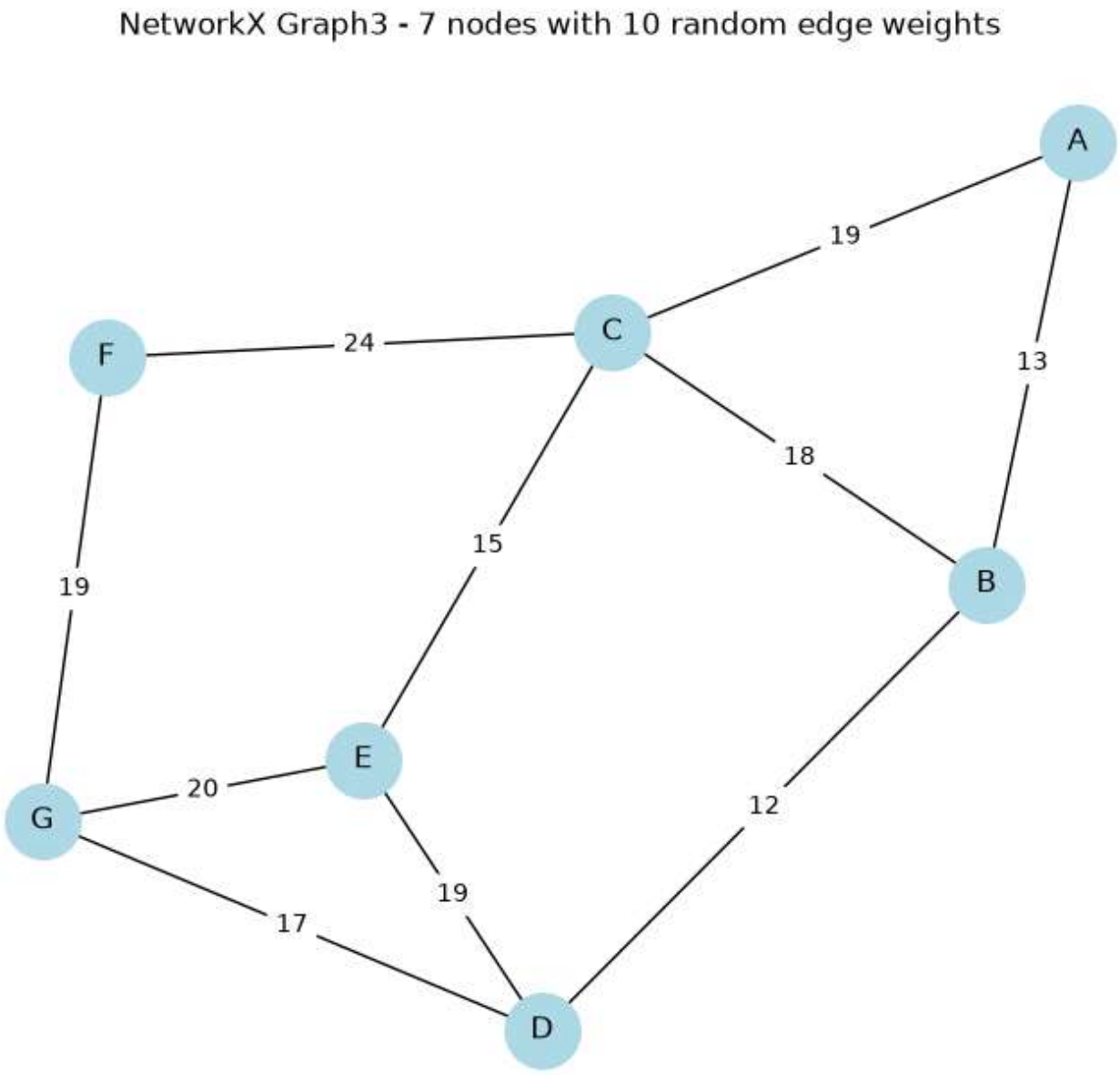
weight3 = nx.get_edge_attributes(graph3, "weight")
nx.draw_networkx_edge_labels(graph3,pos3,edge_labels=weight3,rotate=False)

plt.title("NetworkX Graph3 - 7 nodes with 10 random edge weights")
plt.show()
```

```
Total nodes: 5
Total edges: 8
Is the graph connected?: True
```




```
Total nodes: 7
Total edges: 10
Is the graph connected?: True
```



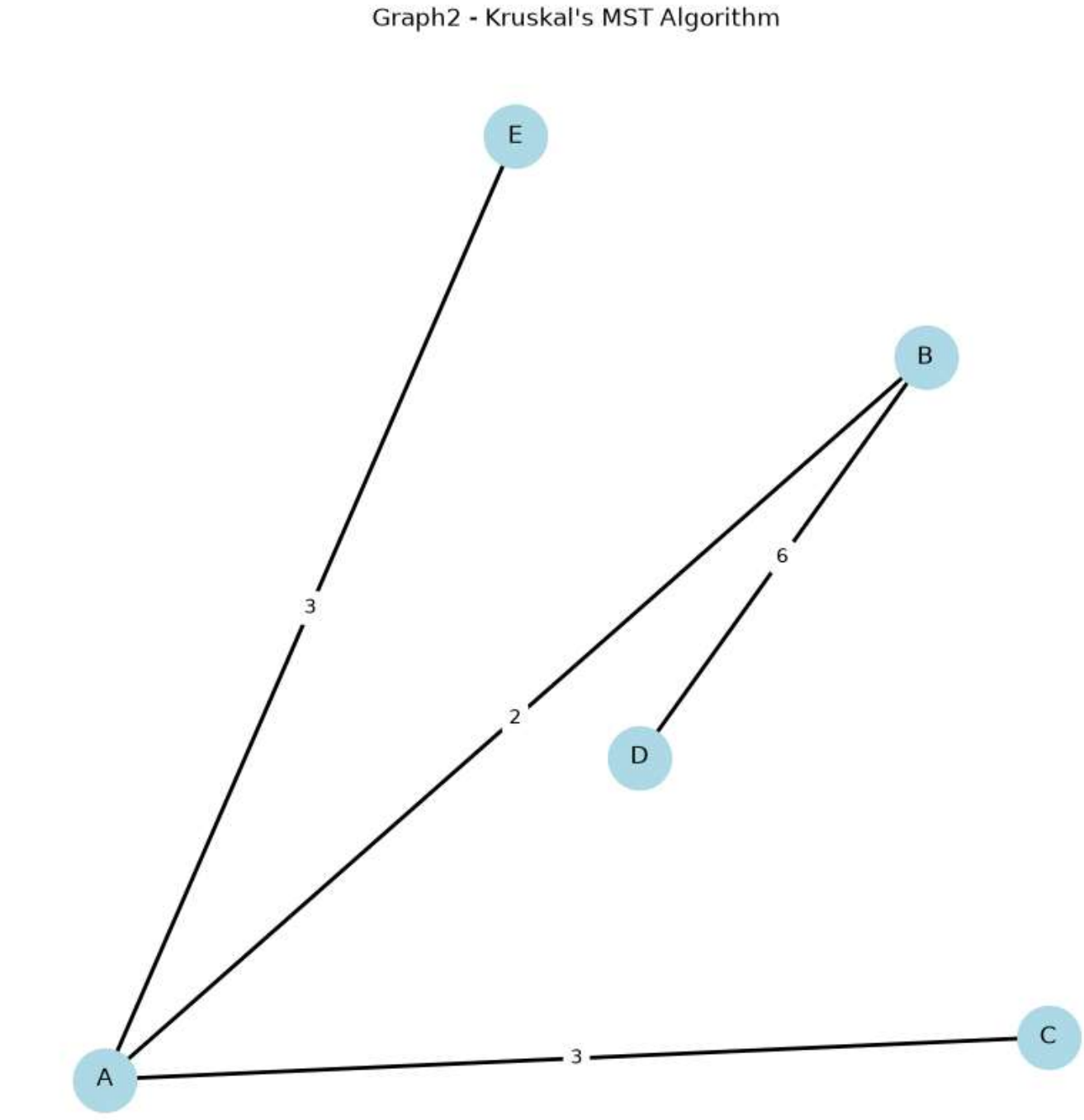
Execute Kruskal's MST algorithm for Graph2 and Graph3:

In [77]:
```
#Graph2 MST
print("Graph2 MST Steps:")
kruskal_result2 = MyMinimumSpanningTree(graph2)
#Generate plot figure
plt.figure(figsize=(8, 8))
nx.draw(kruskal_result2,pos2,with_labels=True,node_color="lightblue",node_size=1000,width=2)
kruskal_weights2 = nx.get_edge_attributes(kruskal_result2, "weight")
nx.draw_networkx_edge_labels(kruskal_result2,pos2,edge_labels=kruskal_weights2,rotate=False)
plt.title("Graph2 - Kruskal's MST Algorithm")
plt.show()

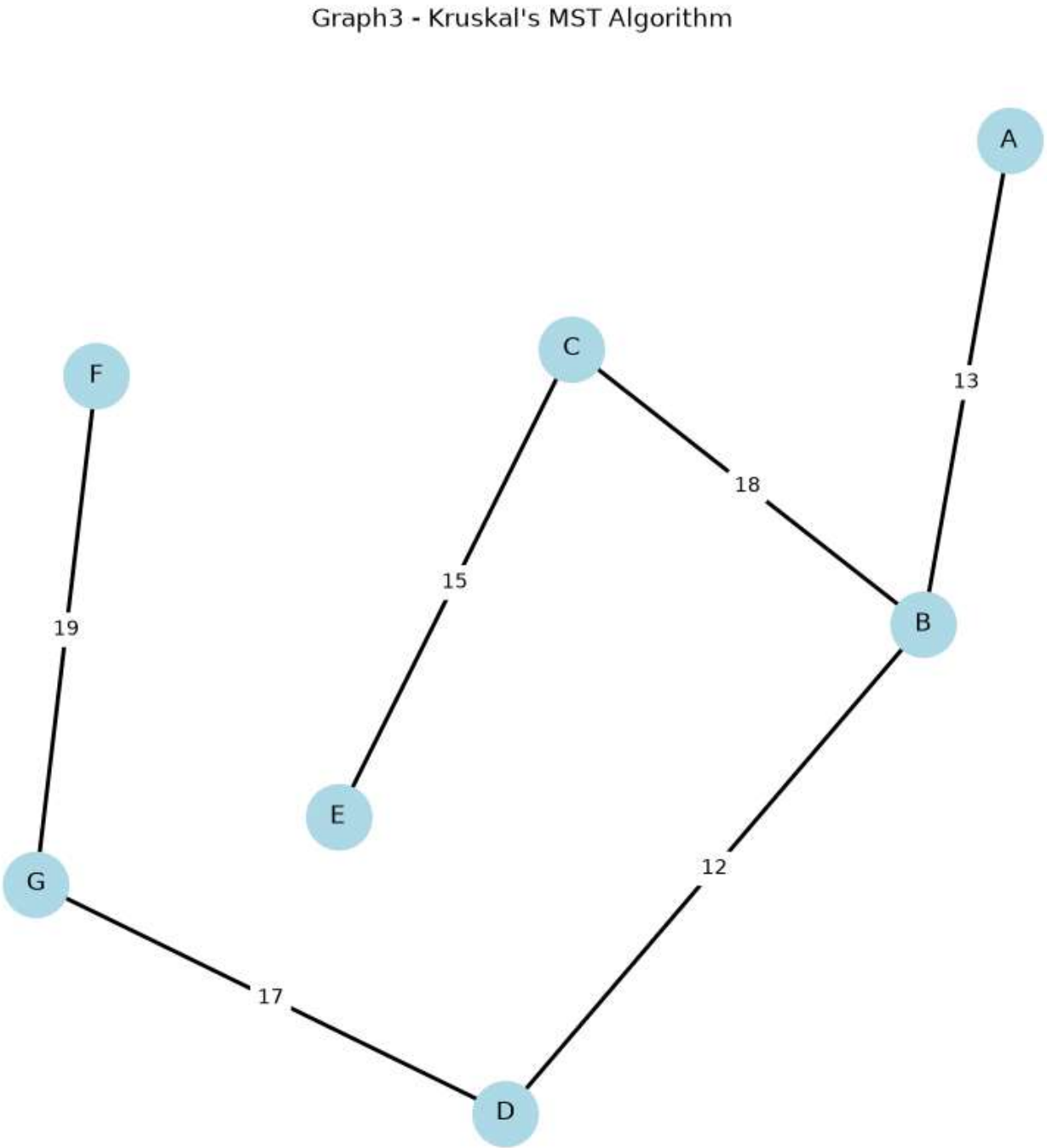
#Graph3 MST
print("\n")
print("Graph3 MST Steps:")
kruskal_result3 = MyMinimumSpanningTree(graph3)
#Generate plot figure
```

```
plt.figure(figsize=(8, 8))
nx.draw(kruskal_result3,pos3,with_labels=True,node_color="lightblue",node_size=1000,width=2)
kruskal_weights3 = nx.get_edge_attributes(kruskal_result3, "weight")
nx.draw_networkx_edge_labels(kruskal_result3,pos3,edge_labels=kruskal_weights3,rotate=False)
plt.title("Graph3 - Kruskal's MST Algorithm")
plt.show()
```

```
Graph2 MST Steps:
Step 1: A -> B with weight: 2 (add)
Step 2: A -> C with weight: 3 (add)
Step 3: A -> E with weight: 3 (add)
Step 4: B -> D with weight: 6 (add)
Step 5: E -> D with weight: 9  (remove)
Step 6: C -> D with weight: 10  (remove)
Step 7: B -> C with weight: 12  (remove)
Step 8: B -> E with weight: 22  (remove)
```



```
Graph3 MST Steps:
Step 1: B -> D with weight: 12 (add)
Step 2: A -> B with weight: 13 (add)
Step 3: C -> E with weight: 15 (add)
Step 4: D -> G with weight: 17 (add)
Step 5: B -> C with weight: 18 (add)
Step 6: A -> C with weight: 19  (remove)
Step 7: D -> E with weight: 19  (remove)
Step 8: F -> G with weight: 19 (add)
Step 9: E -> G with weight: 20  (remove)
Step 10: C -> F with weight: 24  (remove)
```



Source citation (accessed 5 November 2026):

Wayne, R. S. a. K. (n.d.-b). Minimum spanning trees. https://algs4.cs.princeton.edu/43ms

networkx.algorithms.shortest_paths.generic — NetworkX 3.7 documentation. (n.d.).
https://networkx.org/documentation/stable/_modules/networkx/algorithms/shortest_paths/generic.html#has_pa

Graph—Undirected graphs with self loops — NetworkX 3.7 documentation. (n.d.).
https://networkx.org/documentation/stable/reference/classes/graph.html

draw_networkx_edges — NetworkX 3.7 documentation. (n.d.).
https://networkx.org/documentation/stable/reference/generated/networkx.drawing.nx_pylab.draw_networkx_ed

Task 3 Recursion algorithms:

Create a function that receives a list of strings. Each string can be either "blue", "red" or "green".
The goal is to move through the list incrementing the index by following the instructions below:
If the string at index i is "blue", the next element is the element following the next appearance of the string "blue".
If the string at index i is "red", the next element is the element in the index 2x(i) + 1.
If the string at index i is "green", the next element is the following element in the list, i.e. at the index i+1

Task3 a: Code the function using recursion

In [80]:
```python
def follow_instructions(instructions: list[str]) -> list[int]:

    #recursive function to check the list
    def index_checker(i):

        #create a stop function as requirement of recursive function
        if i >= len(instructions):
            print("stop condition envoked - index greater or same as instruction list")
            return []

        #Check current colour at the index i
        current_colour = instructions[i]
        print(f"current index {i}: and current colour: {current_colour}")

        #If the string at index i is "blue", the next element is the element following the next
        if current_colour == "blue":
            print(f"current colour: {current_colour}, apply rules for blue")
            try:
                #find the next "blue" index then return the index after that element
                blue_next = instructions.index("blue", i + 1)
                find_index = blue_next + 1
                print(f"next index: {find_index}")
            except ValueError:
                print("No further blue colours found")
                return[i]
```

```
        #If the string at index i is “red”, the next element is the element in the index 2x(i) +
        elif current_colour == "red":
            print(f"current colour: {current_colour}, apply rules for {current_colour}")
            find_index = (2 * i) + 1

        #If the string at index i is “green”, the next element is the following element in the l
        else:
            print(f"current colour: green, apply rules for green")
            find_index= i + 1
            print(f"next index: {find_index}")

        #Return current and new index based on the conditions
        return[i] + index_checker(find_index)

    return index_checker(0)
```

```
current index 0: and current colour: blue
current colour: blue, apply rules for blue
next index: 3
current index 3: and current colour: blue
current colour: blue, apply rules for blue
next index: 6
current index 6: and current colour: red
current colour: red, apply rules for red
stop condition envoked - index greater or same as instruction list
Final result: [0, 3, 6]
```

Test with two lists:

list1=["blue", "green","blue","blue","red","blue","red", "blue", "green", "red"]

list2=["green","green", "green","blue","blue","red","blue","red", "blue", "green", "red","blue", "red", "red"]

In [82]:
```
list1=["blue", "green","blue","blue","red","blue","red", "blue", "green", "red"]
list2=["green","green", "green","blue","blue","red","blue","red", "blue", "green", "red","blue",

print("Execute on list1:")
result_list1 = follow_instructions(list1)
print("Results result_list1:", result_list1)
print("\n")
print("Execute on list2:")
result_list2 = follow_instructions(list2)
print("Results result_list2:", result_list2)
```

```
Execute on list1:
current index 0: and current colour: blue
current colour: blue, apply rules for blue
next index: 3
current index 3: and current colour: blue
current colour: blue, apply rules for blue
next index: 6
current index 6: and current colour: red
current colour: red, apply rules for red
stop condition envoked - index greater or same as instruction list
Results result_list1: [0, 3, 6]


Execute on list2:
current index 0: and current colour: green
current colour: green, apply rules for green
next index: 1
current index 1: and current colour: green
current colour: green, apply rules for green
next index: 2
current index 2: and current colour: green
current colour: green, apply rules for green
next index: 3
current index 3: and current colour: blue
current colour: blue, apply rules for blue
next index: 5
current index 5: and current colour: red
current colour: red, apply rules for red
current index 11: and current colour: blue
current colour: blue, apply rules for blue
No further blue colours found
Results result_list2: [0, 1, 2, 3, 5, 11]
```

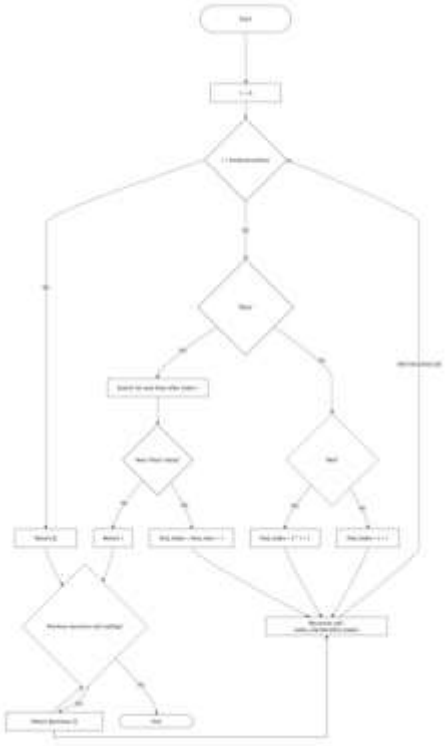
Flowchart of recursive function:

The image below of the flowchart is compressed, for a larger and more clear view, please view the flowchart image in the Git directory:

https://github.com/mjvdheever/DSA2_Coursework/blob/main/Flow3.jpg

In [83]:
```
import matplotlib.image as mpimg
Flow3 = mpimg.imread('Flow3.jpg')
plt.axis('off')
plt.imshow(Flow3)
```

Out[83]:
```
<matplotlib.image.AxesImage at 0x2616d02da90>
```



Task 4 Needleman-Wunsch:

Use the optimal alignment algorithm with the strings ATTACACG and CACAGAGT

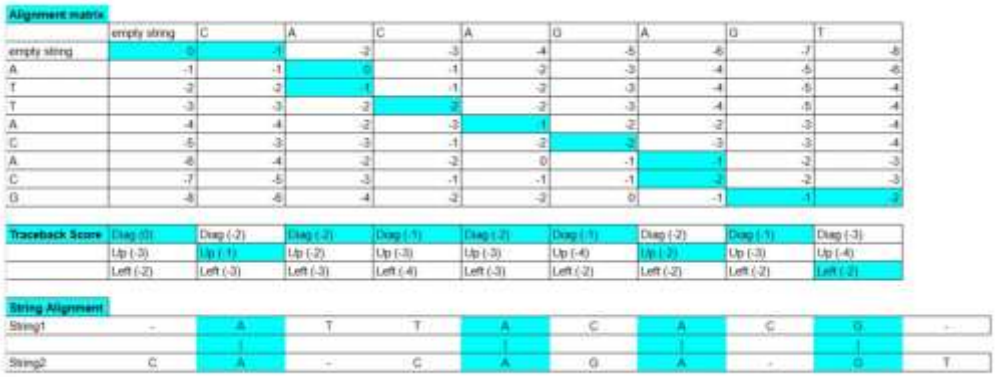
String1: "ATTACACG"
String2: "CACAGAGT"

Unit 6.1 scoring:
Match = 1 point
Mismatch = -1 point
Gap = -1 point

Alignment matrix, traceback scores and string alignment

In [89]:
```
import matplotlib.image as mpimg
am = mpimg.imread('alignment matrix.jpg')
plt.axis('off')
plt.imshow(am)
```

Out[89]:
```
<matplotlib.image.AxesImage at 0x2616c4d4e10>
```



Assume substituition penalty > gap penalty:
Match = 1 point
Gap = -1 point
Substituition = -3 points

In [90]:
```
import matplotlib.image as mpimg
sub_penalty = mpimg.imread('Sub penalty.jpg')
plt.axis('off')
plt.imshow(sub_penalty)
```

Out[90]:
```
<matplotlib.image.AxesImage at 0x2616cc1d310>
```



Task 5 Unit 6.2:

Question 6.2.a:

Cat Swarm Optimization:
Tracking behaviour: Consiously following and movings towards a target, moving according to it's own velocities for every dimension. With each move, it updates it's own velocity and position towards the best solution identified up until that point

Seeking behaviour: Resting behaviour, while aware of surroundings, may or may not move to a different location.
The are four factors to consider in seeking behaviour:
Seeking memory pool (SMP): Define the size of the seeking memory for each cat.
Seeking range of the selected dimensions (SRD): Declares the mutative ratio for the selected dimensions.
Counts of dimension to change(CDC) : Indicates how many dimensions will be varied.
Self-position considering (SPC): Boolean variable, which decides whether the point the cat is already standing at will be one of the candidates to move to

Grey Wolf Optimization:
The algorithm creates wolves into four groups, Alpha, Beta, Delta and Omega, where the Alpha wolf is the best solution indetified so far and Omega wolf is the all other remaining solutions after Alpha, Beta and Delta wolves.

Approaching the prey:
Search according to the position of the alpha, beta and delta. Diverge from each other to search for prey and then converge to attack prey. Essentially exploring multiple different possible solutions instead of focusing on one singular location

Encircling the prey:
Guided by the Alpha (best solution identified so far), the Beta and Delta wolves will adjust their position to the Alpha, converging to the best possible solution identified at this point.

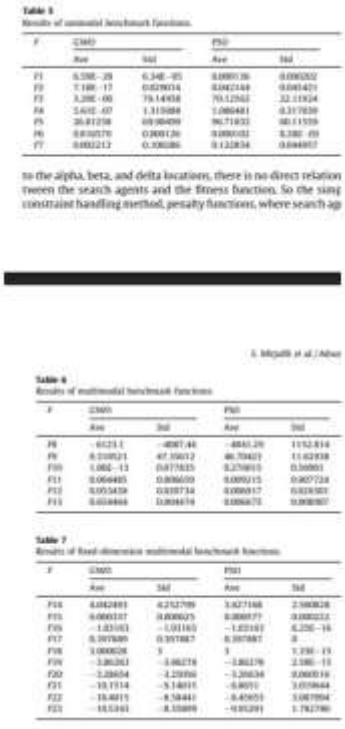
Attacking the prey:
As progressing, the algorithm/wolves focus more closely on the best solution identified so far, it will attempt improve best idenitified solution instead of keep searching for other solutions. The is finished when prey is attacked and when the algorithm/wolves stop moving.

Question 6.2.b:
Both Cat Swarm Optimization and Grey Wolf Optimization use the pre-existing **Particle Swarm Optimization (PSO)** algorithm.

Based on the performance metrics provided in the paper, Grey Wolf Optimization outperforms PSO, however PSO did outperform Grey Wolf Optimization in a few test function metrics as can be seen in the table below:

In [92]:
```
import matplotlib.image as mpimg
GWOvsPSO = mpimg.imread('GWO vs PSO.jpg')
plt.axis('off')
plt.imshow(GWOvsPSO)
```

Out[92]:
```
<matplotlib.image.AxesImage at 0x2616cac3250>
```



Based on the performance metrics provided in the paper, Cat Swarm Optimization outperforms PSO with the weighting of six test functions as can be seen in the graph below:

In [93]:
```
import matplotlib.image as mpimg
CSOvsPSO = mpimg.imread('CSO vs PSO.jpg')
plt.axis('off')
plt.imshow(CSOvsPSO)
```

Out[93]:
```
<matplotlib.image.AxesImage at 0x2616c3f2490>
```

4 Experimental Results

We applied CSO, PSO and PSO with weighting factor into six test functions to compare the performance. All the experiments demonstrate the proposed Cat Swarm Optimization (CSO) is superior to PSO and PSO with weighting factor. Due to the space limit of this paper, only the experimental results of test function one shown in Fig. 1.

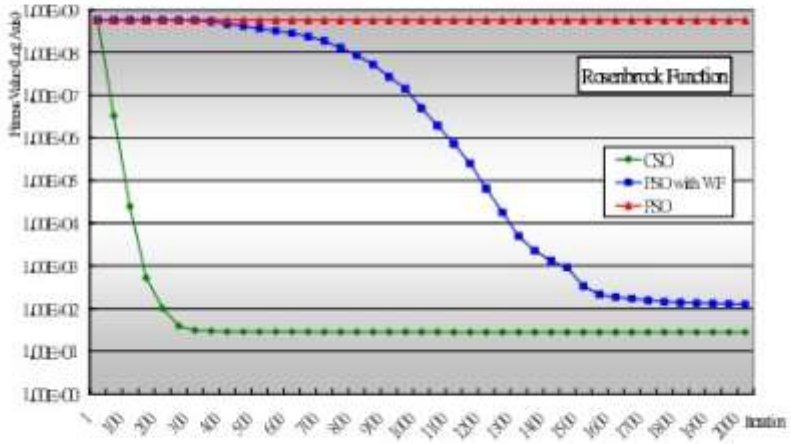


Fig. 1. The experimental result of test function 1